

DAR ES SALAAM INSTITUTE OF TECHNOLOGY

COURSE: COMPUTER ENGINEERING

NAME: IRENE BHOKE MATHIAS

REG NO: 240222481166

CLASS: OD24COE

MODULE NAME: FUNDAMENTAL OF DATA STRUCTURE
AND ALGORITHMS

MODULE CODE: ITT 05213

INDIVIDUAL ASSIGNMENT 01

LECTURE, DANIEL MADUTU

Qn:1 Rank Complexities (slowest - fastest).

Rank	Complexity	Description.
1 (slowest)	$O(1)$	- Constant time - does not grow with input
2	$O(\log n)$	- Logarithm - doubles input, + 1 step
3	$O(n)$	- Linear - grows proportionally with n
4	$O(n \log n)$	- Linearithmic - common in efficient sorts.
5 (fastest)	$O(n^5)$	- polynomial - grows very rapidly.

Qn:2 Big-O Complexity analysis

- for each function, drop all constants and lower-order terms to find the dominant term.

Part	Function	Dominant term	Big-O
a,	$4n^2 + 20n + 9$	$4n^2$	$O(n^2)$
b,	$25n^3 + 15$	$25n^3$	$O(n^3)$
c,	$n^3 + 5 \log n + 6$	n^3	$O(n^3)$
d,	$10^5 + 10^5 n + 10^5 n^2$	$10^5 n^2$	$O(n^2)$
e,	$72n(n + 3n^2) = 72n^2 + 216n^3$	$216n^3$	$O(n^3)$
f,	$2^8 + 324n - 5$	$324n$	$O(n)$

Qn:3 Algorithm to sum 1 to n in $O(n)$.

Sum_to_n (n ; integer) : integer

begin:

total := 0

for $i := 1$ to n do:

total := total + i

end for

return total

end.

Qn: 04 Code Fragment Analysis.

① For this case, give the Complexity in terms of the variable number

$X = 4$

for $i := 0$ to number do:

$X := X + 1$

end for

+ $O(1)$

+ number iterations

+ $O(1)$ body.

$\therefore$ Complexity = $O(\text{number})$.

⑤ In the following assume that the function get-choice() runs in $O(1)$ time, and give the Complexity in terms of the cycles variables.

get-choice() runs in $O(1)$.

If Choice = 2, the loop runs from 0 to cycles.

Worst-case complexity: $O(\text{cycles})$. $\therefore$ Complexity = $O(\text{cycles})$.

⑥ There are two functions below. Find the complexity of the calculate function.

increase(val, iter) runs in $O(\text{iter})$ because of one loop.

calculate calls increase inside another loop running iter times.

$\therefore O(\text{iter} \times \text{iter}) = O(\text{iter}^2)$

$\therefore$ Complexity = $O(\text{iter}^2)$.

⑦ Complexity of Calculate function.

Analysis;

- Increase(val, i): cost is $O(i)$ per call (not iter - passes it as second arg)

Total cost: $O(1) + O(2) + \dots + O(\text{iter}) = O(\text{iter}(\text{iter}+1)/2)$

$= O(\text{iter}^2)$

$1 + 2 + \dots + n = n(n+1)/2$ (uses the natural number summation formula.)

$\therefore$ The complexity is $O(\text{iter}^2)$.

$\therefore$ The complexity is $O(\text{iter}^2)$.

Qn:05 i) If $f(n)$ is $O(g(n))$,
then, $f(n) \leq C g(n)$.

ii) If $f(n)$ is $\Omega(g(n))$,
then, $f(n) \geq C g(n)$.

Qn:06 Show that $4n+12$ is $O(n)$

Soln

We need constants C and K such that:

$$4n+12 \leq Cn$$

$$\text{Let } C=16 \text{ and } K=1$$

For $n \geq 1$:

$$4n+12 \leq 16n$$

$$= 4n+12 \text{ is } O(n).$$

$\therefore$ Therefore, $4n+12$ is $O(n)$. $\#$

Qn:07 Show that $5n^2-16$ is $O(n^2)$

Soln

We need constants C and K such that:

$$5n^2-16 \leq Cn^2$$

$$\text{Let } C=5 \text{ and } K=1$$

For $n \geq 1$:

$$5n^2-16 \leq 5n^2$$

$\therefore$ therefore, $5n^2-16$ is $O(n^2)$.